CS 2110 Homework 7
Dynamic Memory and Malloc Implementation

Elizabeth Hong, Henry Bui, Biana Jayaraman, Owen Anderson,
Iris Song, Ganning Xu, Aadit Bansal, Bryan Shook, Joseph Seo

Spring 2025

Contents

1 Overview

1.1 Purpose

The purpose of this assignment is to introduce you to dynamic memory allocation in C and then give you a deeper understanding of how malloc works at runtime.

You will learn how to manage memory in C with the malloc family of functions, and then you will implement this family of functions yourself.

2 Part 1 Instructions

2.1 Implementation Overview

While your last Minecraft game was a huge success, you've recently learned about The Heap and dynamic memory allocation and you want to try your hand at making it. Additionally, your game has received some reviews that claim it is too complicated, so some of your previous functions need to be simplified (ex. there are no more Items).
Your Minecraft world will be represented by a `struct world`. Each world has an array of players in the world and a linked list of monsters in the world. Each player has their own name and level. The linked list is composed of Nodes that have a monster and a next node. The monster has an enum representing its type as a level. Player and monster have their own structs. You can find details about these structs and more in the included **minecraft_v2.h** file, as well as in this document.

You will be writing code in

1. minecraft_v2.c
2. main.c

(main.c is only for your own testing purposes, you will not submit it.)

Be sure not to modify any other files. The changes in the other files will not be reflected by the autograder, so you will not be able to use them.

Remember that the structs passed in to functions are already malloc-ed.

Forgetting to free this data when it is removed can cause memory leaks (memory that's no longer being used but not freed), but freeing it when it should be returned to the user can created dangling pointers (memory that's freed but still being referenced). Keep this in mind when writing functions that deal with removing elements.

You will create and allocate the following structs: `struct world`, `struct Player`, `struct Monster`, `struct LinkedList`, and `struct Node`. Below is an overview of what each struct contains.

Each `struct World` has the following:

- `LinkedList monsters`, a linked list of all the monsters in the world.
- `Player **players`, a pointer to an array that holds pointers to `struct Player`s. You will need to dynamically allocate space for this array.
- `int num_players`, the number of players in the world

Each `struct Player` has the following:

- `char *name`, a pointer to the name of the player.

- `int level`, the Player's level.

Each `struct Monster` has the following:

- `enum Mob mob`, an enum denoting the type of monster.
- `int level`, the Monster's level.

Each `struct LinkedList` has the following:

- `Node *head`, the head of the Linked List.
- `int size`, the number of nodes in the Linked List currently.

Each `struct Node` has the following:

- `struct Node *next`, a pointer to the next `Node` in the list.
- `Monster *monster`, a pointer to a `Monster` that the node represents in the Linked List.

Useful Macro Definitions in `minecraft_v2.h`:

- `SUCCESS` is an macro for the integer value to return when a function operation succeeds.
- `FAILURE` is an macro for the integer value to return when a function operation fails. Think of this as an error code.
- `INCOMPLETE` is used as the default return value if you have not completed a function.
- `UNUSED_PARAM(x)` is a macro that is used in `minecraft_v2.c` as a placeholder so that you can compile the file without needing to complete every function. You may remove these once you've completed a function.

2.2 `minecraft_v2.c`

The premise of Minecraft version 2 is similar to Homework 5's Minecraft version 1. However, there are some key differences. Given that the purpose of this homework is to familiarize you with dynamic memory, and less so with C syntax and logic, much of the implementation is focused on creating, modifying, and removing structures. Therefore, while you have some of the same structs, their content has been changed to accommodate for dynamic memory (ex. the name for Player is now dynamically allocated rather than being a set size), and some of the previous functions you implemented have been simplified as they have little to do with dynamic memory (such as `fight_monster`).

The methods you will be implementing are as follows:

- `World *create_world(void)`

  You can now have more than one world! This function should create a world and initialize each member of the struct to it's default value.

  - Initialize each pointer to be `NULL` and each integer to be 0.
  - Initializes monsters' head to NULL and sets monsters' size to 0.
  - If malloc fails, return `NULL`.
  - Otherwise, return the pointer to the world you just created.

- `int create_player(World *world, const char *name, int level)`

  You want to be able to have players in each world. This function should create a new player.

  - The name of the player should be unique to the world (there might be another function that can help you determine this). If it is not already being used, assign it and the level to the player.
  - The player should be added to the end of the World's player array. Update the world's attributes as necessary.
  - Resize the backing array to only accomodate for the new player (i.e., only add enough space for one more player)
  - If malloc fails, the player name is not unique, the level inputted is less than or equal to 0, or any of the pointer parameters are `NULL`, return `FAILURE`.
  - Otherwise, return `SUCCESS`.

- `int create_monster(World *world, enum Mob mob, int level)`

  You want to be able to have monsters in each world. This function should create a new monster.

  - You should create a new Monster and place it in a Node. This node should be added to the end of the world's linked list of Monsters (what should happen if the list is empty?).
  - The monster should be added to the end (tail) of the World's monster linked list. Update the linked list's attributes as necessary as well.
  - If malloc fails, the level inputted is less than or equal to 0, or any of the pointer parameters are `NULL`, return `FAILURE`.
  - Otherwise, return `SUCCESS`.

- `Player *find_player(World *world, char *name)`

  You may want to find a player within a world (Hint: this could be helpful in other functions).

  - Return the pointer to the player in the given world that has the passed-in name.
  - If any of the pointer parameters are `NULL`, or if the player isn't in the world `NULL`.
  - Otherwise, return the pointer to the player.

- `int fight_monster(World *world, char *name)`

  A player may want to fight some of the monsters in the world for glory and aura.

  - The monster the player fights should be the first one in (the head of) the linked list of monsters in the world.
  - The player fighting should have the passed in name.
  - The fight should be determined by the level of the player and monster. If the player's level is greater than or equal to the monster's level, the player wins. Otherwise, the monster wins.
  - If the player wins:
    - Delete the monster from the list. Update the world attributes and linked list as necessary.
    - Increment the player's level by 1.
  - If the monster wins:
    - If the monster is a creeper, the player's level should be reset to 1 and the creeper's level should be incremented by 1.
    - If the monster is not a creeper, the player should be deleted and the monster's level should be incremented by 1.
  - If any of the parameters are `NULL`, there are no monsters in the world, the player with the passed in name does not exist, return `FAILURE`. Otherwise, return `SUCCESS`.

- `int delete_player(World *world, char *name)`

  Sometimes players die. In such cases, they need to be deleted from the world.

  - The name passed in is the name of the player to be deleted from the world.
  - The world's player array should remain contiguous after the player has been deleted.
  - Everything associated with the player should be deleted as well.
  - If any of the parameters are `NULL`, or if the player does not exist in the world, return `FAILURE`, otherwise, return `SUCCESS`

- `int delete_world(World *world)`

  You may want to delete a world (bad spawn). This function achieves that.

  - Everything associated with the world should be deleted. This includes everything within each member of the world struct.
  - If the world passed in is `NULL`, return `FAILURE`, otherwise, return `SUCCESS`

Once you've implemented the functions in the minecraft_v2.c file, or after implementing a few, compile your code using the Makefile. You may test your functions manually by writing your own test cases in the provided `main.c`, or run the autograder. See the Testing section below for more information.

Please COMPILE OFTEN. The compiler will reveal many syntax errors that you would rather find early before making them over and over throughout your code. Try to write and test your functions incrementally as well, as some will rely on others. Waiting until the end to compile for the first time will cause big headaches when you are trying to debug code. We speak from experience when we say compile often. :)

2.3 Implementation Tips

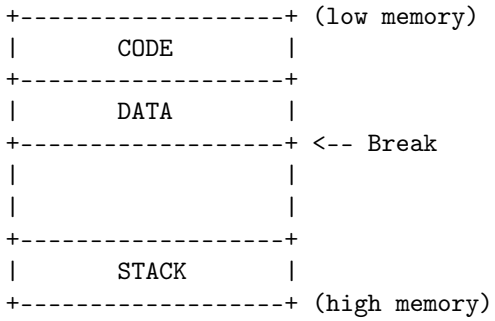
- When creating structs, consider what parts of the struct need to be dynamically allocated with the use of malloc. We use dynamic allocation when amount of space needed to be allocated is unknown at compile time, so think about which members of the structs do not have a constant size.
- calloc and realloc are your friends. Using them instead of malloc in certain cases will help make your life easier and reduce the amount of code you'll need to write.
- When freeing structs, you will need to free any components of the struct that have been malloc-ed individually before you can free the struct itself. Keeping this in mind will help prevent memory leaks in your code.

3 Part 2 Instructions

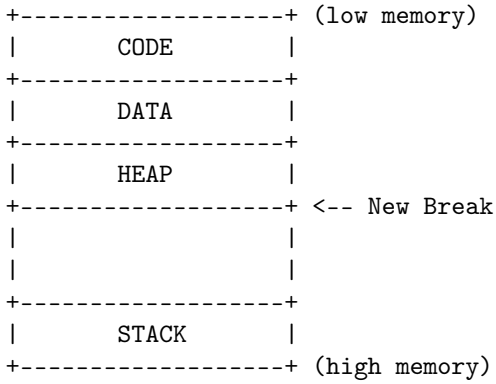
In this assignment, you will be writing the dynamic memory allocation and deallocation functions of malloc, free, realloc, and calloc. These functions are confusing to write, so we have provided an in-depth guide below. **Please read through this entire section before beginning.** The specifics for each function are located in `malloc.c` as well as in the subsections below.

3.1 The Basics

It is the job of the memory allocator to process and satisfy the memory requests of the user. But where does the allocator get its memory? Let us recall the structure of a program's memory footprint.



You've learned in lecture and lab that a program has various "segments" that serve different purposes: code, stack, data, etc. In order to create some dynamic memory space, otherwise known as the heap, it is possible to move the end of the process's uninitialized data segment, or the "break," effectively giving the program more memory. There is a function called `sbrk()` which moves the break by the amount specified and returns a pointer to the beginning of the new memory block.



For simplicity, a wrapper for the system call `sbrk()` has been provided for you as a function called `my_sbrk` located in `suites/malloc_suite.c`. Similar to `sbrk`, `my_sbrk` returns a pointer to the beginning of the requested memory. **Make sure to use this call rather than a real call to sbrk, as doing a real call to sbrk can potentially cause a lot of problems during program execution.** Note that any problems introduced by calling the real sbrk will not be regraded.

We've written `my_sbrk` to be able to only hand out a certain amount of memory before returning -1 to indicate that it's done. This limit gives us the ability to test the behavior of the code when `my_sbrk` can't get more memory.

3.2 Block Allocation

Trying to use `sbrk()` (or `my_sbrk`) exclusively to provide dynamic memory allocation to your program would be very difficult and inefficient. Calling `sbrk()` involves a decent amount of system overhead, and we would prefer not to have to call it every single time a small amount of memory is required. In addition, deallocation would be a problem. Say we allocated several 100 byte chunks of memory and then decided we were done with the first. Where would the break be? There's no handy function to move the break back, so how could we reuse that first 100 byte chunk?

What we need are a set of functions that manage a pool of memory allowing us to allocate and deallocate efficiently. In order to keep track of allocated blocks we will create a structure to store the information we need to know about a block. Where should we store this structure? We can't call `malloc()` to allocate space for our malloc information structure, since we're writing the function, and that'd lead to infinite recursion! However, there's an easier way that will keep our bookkeeping structure right with the data we're allocating for easy access.

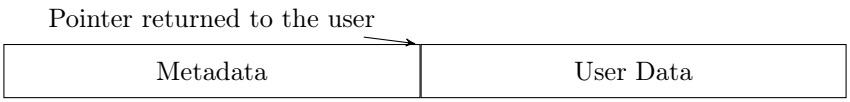
In order to keep track of allocated blocks, we will create a structure to store the information about a block (metadata) within the block itself. The metadata contains two things: a pointer to the next node in the freelist, and the size of the user data section. Both of these are required in order to accurately keep track of the memory available in the freelist.

Diagram of an allocated block of memory. The metadata is placed before the user data. The next byte after the end of the metadata is the first byte that can be given to the user.

We will need to take into consideration the leading metadata whenever we allocate blocks. To let the user have as much space as they requested, when they request a block of size **n** bytes we must actually allocate a block of size `sizeof(the metadata) + n`. The metadata will contain the size of the memory available to the user (n) and a pointer to the next metadata struct in the freelist. As depicted in `my_malloc.h`, this is the struct definition for the metadata:

```
typedef struct metadata {
    struct metadata *next_size;
    unsigned long size;
} metadata_t;
```

The size portion of the metadata struct contains the size that the user requested. To get the total size of the block, we add `size` with `sizeof(metadata_t)`, which is the size in bytes of the metadata. For ease of reading, this size will be represented as `TMS` (total metadata size, defined as `TOTAL_METADATA_SIZE` in your homework files) in all of our block representation diagrams. The user does not care about the metadata for the block, they just want the size they requested. **Therefore, when you return a block to the user, you will need to use *pointer arithmetic* to step over' the metadata and return the address of the data.** What this looks like:



When a block is returned to the user, the pointer returned points to the beginning address of the area used by the user.

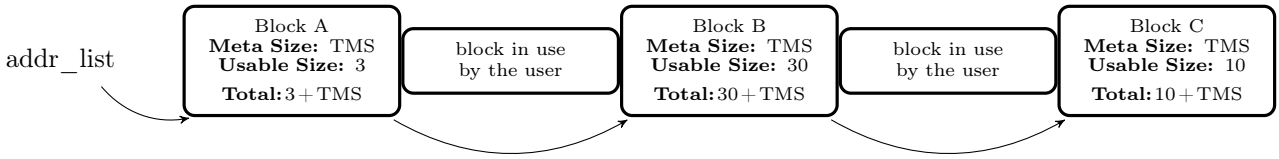
3.3 The Freelist

When we split up memory, we give one piece/block to the user. The remaining pieces/blocks are placed in a linked list, called the freelist, to be used at a later time. For this semester, we are representing our freelist as

a single singly linked list that is organized by the address in ascending order. This linked list will be defined as a global file variable and to help you out, we have already defined it for you.

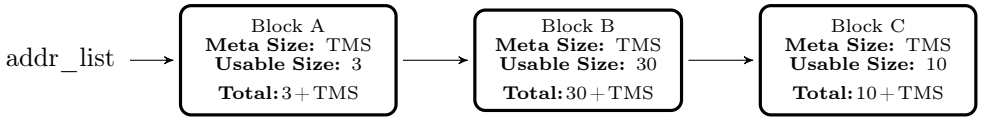
```
metadata_t *addr_list;
```

To help visualize this, below we have an example representation of our freelist.



Note: The name of the blocks refer to their address order. For example, since the letter B comes after the letter A, Block B starts at an address after Block A.

For the remainder of the pdf, we will represent the freelist without spaces for the blocks currently in use by the user like so:



A Quick Note: The node representations in our freelists should be read as the following:

1. First Line: The name of the block ("Block B") - Note that the name refers to the ordering that the blocks should be in.
2. Second Line: **Meta Size** → The size of the metadata for that block
3. Third Line: **Usable Size** → The size of the space available to the user
4. Fourth Line: **Total** → The total size of the memory taken up by this block

Since the addr_list is singly linked, be sure to properly update the next pointer when adding and removing nodes from the list.

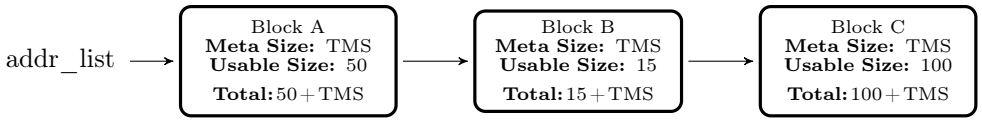
3.4 Simple Linked List: Allocating

When we first allocate space for the heap, it is useful to request more than we need and keep the extra around in the freelist until we need it. This reduces the amount of times we need to call `my_sbrk()`, the real version of which, as we discussed earlier, involves significant system overhead. So how do we know how much to allocate, how much to give to the user, and how much to keep?

For this assignment we will request blocks of size 2048 bytes from `my_sbrk()`. We don't want to waste space, though, so we want to give to the user the smallest size block in which their request would fit. For example, the user may request 256 bytes of space. It is tempting to give them a block that is 256 bytes, but remember we are also storing the metadata inside the block. If our metadata takes up `sizeof (metadata_t) = 16` bytes for example, we need at least a $256 + 16 = 272$ byte block.

How do we get from one big free block of size 2048 bytes to the block of size 272 bytes we want to give to the user? In this simple implementation, you will traverse the `addr_list` to find the best block to satisfy the user's request, which should be equal or greater than the size requested, and "split" off however much you need from the front or the back. For this assignment, you must split off from the back.

Say we have the following situation:



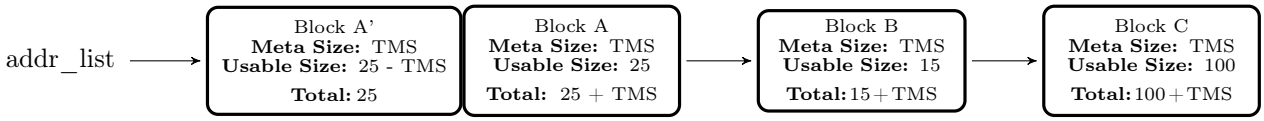
When we malloc for a certain size, we first want to use a block of that exact or best size, remove it from both the `addr_list` and return it to the user.

Ex: `malloc(15)` would leave the freelist as so:

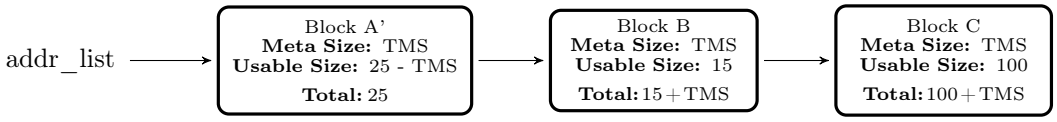
If we do not have a perfectly sized block, then we will return the smallest block in the list that has enough room to hold the user's requested data. There are actually two cases to consider:

- If a block's user data size is at least `requested_size + TMS + 1`, then you should split the block into two halves. The right split should become its own block, with its own metadata and `requested_size` bytes in the user data section. This is the block that will eventually be returned to the user. We need to update the left split to reflect its new user data size of `old_size - TMS - requested_size`. Note that the user size is still at least 1 byte, making it a valid block. This block should remain in the freelist.
- If a block's user data size is smaller than `requested_size + TMS + 1`, then splitting the block would not leave enough room for the metadata and at least a byte of user data. So, instead of splitting, **return the whole block**, even though the user gets a bit more space then they need.

In the following diagram, the requested size is 25 bytes, and the first block has enough room:



Once Block A is returned to the user, this call will leave the freelist as such:



Don't forget to move the pointer to the beginning of the space the user uses at the end of the metadata before returning the block to the user.

3.5 Simple Linked List: Deallocating

When we deallocate (free) memory, we simply move the now freed block back to the `addr_list` in the appropriate position. When the user calls the free function with a block body pointer, we do some pointer arithmetic to find the starting point of the entire block (i.e. the start of the metadata). Notice we don't zero out or overwrite all the data. That simply takes too long when we're not supposed to care about what's in memory after we free it anyway. For all of you who were wondering why sometimes you can still access data in a dynamically allocated block even after you call free on its pointer, this is why!

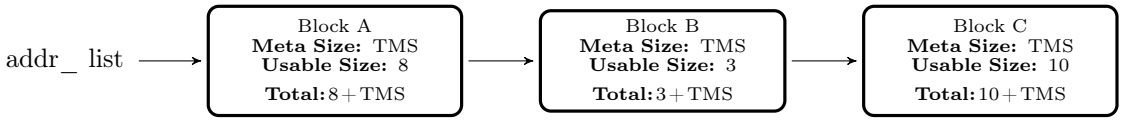
We like the freelists to contain fairly large blocks so that large requests can be allocated quickly, so if the block on either side of the block we're freeing is also free, we can coalesce them, or join them, into the bigger block like they were before we split them.

How do we know which blocks we can join together? If adding a free block's address and the block's total size (both TMS and the user data size) gives you the address of another free block's metadata, then you know that those two blocks are next to each other in memory, so they can be merged. If A has a size of M and B has a size of N, then if they are next to each other in memory, you can merge them to create a new block of size M+N+TMS. Note that the size includes the metadata size because while both A and B had space for metadata, the new combined block only needs space for one metadata struct.

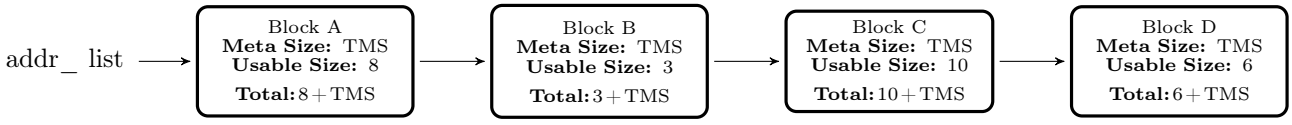
Whenever you deallocate a block, you must merge it with nearby free blocks if possible. There are a couple of ways to do this:

- A simple technique is to search the freelist for blocks that are next to the newly freed block. Conceptually, it looks like this:
  - Given a block B, iterate through the free list and find a block A that is located directly to the left of B in memory. This means that if we take A's address and add A's total size, we line up perfectly with B's address, meaning there are no gaps between A and B. If this block exists, we can merge A/B, then remove A from the freelist.
  - We can do the same procedure for blocks that are located directly to the right of B in memory. Again, if a block C exists, we merge B/C and remove C from the freelist.
  - Finally, we re-insert B or our newly merged blocks back into the freelist. Don't forget to keep nodes sorted by address!
- We can optimize this process by simply adding the newly freed block to the freelist first, then scanning through the list for blocks to merge. Since the freelist is sorted by address, two blocks MUST be next to each other in the freelist in order to be next to each other in memory. This means you can go through every pair of consecutive elements in the freelist, check if the blocks are touching in memory, and merge them if necessary. When merging contiguous blocks, it is recommended to leave the left block in the freelist and grow its size, while removing the right block.

Here's an example of how to free and merge blocks:



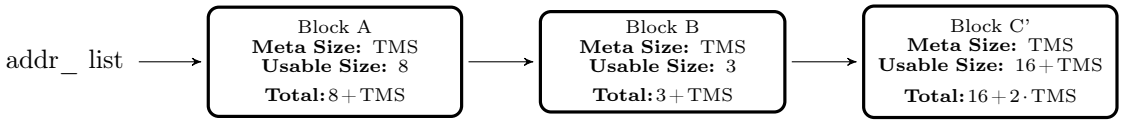
If we deallocated a block of size 6 (Block D), we would first iterate through the `addr_list` for the correct left and right addresses of the block and check to see if the block needs to be merged either to the right or left. In this example, the block to be entered is not directly next to any other blocks in memory, so we would just insert it into the `addr_list`. This would leave the freelist as seen below.



If Block C and D were right next to each other in memory (i.e. the address at end of block C is equal to the address at the beginning of block D), then we would need to perform a left merge. To perform this left merge, pop block C from the `addr_list` , add block D to it, reset the size, and add it to the `addr_list` in the same position that block C was in. Note that this series of steps assumes that you have not yet added D to the freelist. These steps are depicted below.

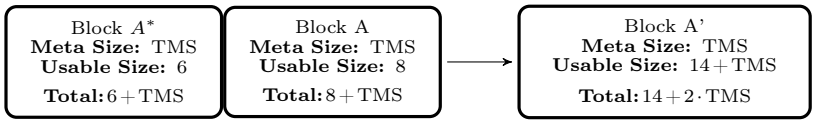
Remove Block C from the free list and merge it with Block D to make Block C'.

Add this new block C' back into the freelist in its proper position.

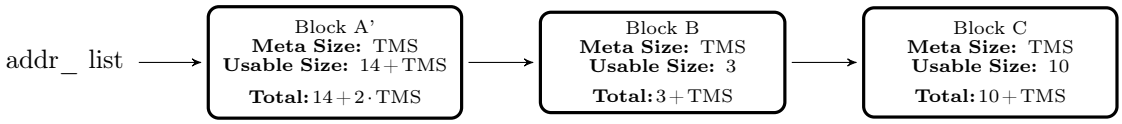


If instead of a Block D we had a Block A^* which was located right before Block A in memory (i.e. the address at the end of block A^* is equal to the address at the beginning of block A), then we would need to perform a right merge. To perform this right merge, pop block A from the `addr_list`, add block A^* to it, move block A's metadata to block A^*, reset the size, and put the block back where block A originally was in the `addr_list`.

Remove Block A from the free list and merge it with Block A^* to form Block A'.



Add this new block A' back into the freelist in its proper position.



Note: To compare pointers (addresses), cast them to char * first

3.6 Helper Methods

Implementing `malloc()` can seem like quite a daunting challenge, but your debugging process can be helped along tremendously if you do not write all of `malloc()` in one method and instead split it up into helper methods! Helper methods are incredibly useful for understanding what is going on and also results in cleaner code, so it's a win-win strategy. Below are some **required** helper methods to implement that will be tested in the autograder, **we advise that you use them for functions discussed in later sections (e.g. `my_malloc()`)**.

Two helper methods are already created for you:

- `metadata_t *find_left(metadata_t*)`
- `void remove_from_addr_list(metadata_t *remove_block)`

Function comments in the homework files will tell you what these methods do (along with the following methods you will be implementing).

The helper methods that you have to implement:

- `metadata_t *find_right(metadata_t*)`
  - Given a pointer to a free block, find a block in the free list that is exactly besides that block in memory, to its right. Note that we are NOT trying to find the next block in the linked list relative to the free block pointer, as that might not be directly next to this free block in memory. Additionally, note that the block directly right of the free block (in memory) may not be in the free list.
- `void merge(metadata_t *left, metadata_t *right)`
  - Given two free blocks that are next to each other in memory, merge them such that the left block now takes up the space of the two existing blocks, and that the right block is essentially removed from the free list.
- `metadata_t *split_block(metadata_t *block, size_t size)`
  - Given a pointer to a free block and a `size` argument, split the block into two such that the right portion has a usable size of `size` bytes (and thus a total block size of `TMS + size` bytes). The left portion of the split should be shrunk accordingly, while the pointer to the right portion is returned. Do not alter the free list.
- `void add_to_addr_list(metadata_t *add_block)`
  - Given a free block pointer, add it to the free list (which is essentially a linked list). Realize that the free list is sorted by the address of the block, so insert the given block accordingly.
- `metadata_t *find_best_fit(size_t size)`
  - Traverses through the linked list to return a free block pointer that best satisfies the best-fit conditions for allocation specified in section **2.4**. Note that you are only to return the pointer to the best-fit free block, DO NOT split this block.

Remember, this is not an exhaustive list of operations that can performed with helper methods. Feel free to implement additional helper methods for any aspect of malloc that works for you.

Note: Typically in production code, these functions would be static because we want them to be private to my_malloc.c. However, in order for the autograder to run your helper functions outside of my_malloc.c, these functions will be declared normally in my_malloc.h and my_malloc.c.

3.7 my_malloc(size_t size)

You are to write your own version of malloc that implements simple linked-list based allocation:

1. The size of the block we are looking for is the size that the user is requesting. (Note: if this size in bytes is over `SBRK_SIZE - TOTAL_METADATA_SIZE`, set `my_malloc_errno` to the error `SINGLE_REQUEST_TOO_LARGE` and return `NULL`
2. If the request size is 0, we do not have to allocate anything; mark `NO_ERROR` and return `NULL`).
3. Now that we have the size we care about, we need to iterate through our freelist to find a block that best fits. Best fit is defined as the first block that has the same user data size as the requested amount of bytes, or the smallest block that has more than enough space for the requested data.
   (a) If the best fit block is exactly the same size, you can simply remove it from the `addr_list` and return a pointer to the body of the block to the user.

(b) If the best fit block is larger than the requested size, but is not big enough to split, then simply remove and return the block, as if it were a perfect fit. To tell if a block is too small to split, take the user data size of the original block and subtract the number of requested bytes. If the remaining number of bytes is not enough to contain a valid block of metadata size + 1 user byte (defined as `MIN_BLOCK_SIZE`), then it is too small to split.

(c) If the block is big enough to house a new block, we need to split off the portion we will use from the right side of the block, keeping the remaining left side in the freelist. Remember: pointer arithmetic can be tricky, so **make sure you are casting to a `char *`** before adding the total size (measured in bytes) to find the split pointer! Otherwise, your calculations might get implicitly multiplied by `sizeof(metadata_t)`.

If no suitable blocks are found at all (i.e., all blocks have a user data size smaller than the requested size), then call `my_sbrk()` with the size `SBRK_SIZE` to get more memory. Our autograder expects exactly this amount of size to be requested, so please make sure to use the macro.

The chunk of memory returned by `my_sbrk()` should be treated as a new block, so you will need to write a metadata_t to the start of the block. This means that the block's available user size is actually `SBRK_SIZE - TMS`.

Additionally, **you must merge this new block with any free blocks that are directly next to the new block in the freelist**. Since the freelist is sorted by address, there is only one block you need to check: the last block in the freelist. You should call `my_free()` to do this.

After setting up the block's metadata and merging it if possible, restart the search for a best-fit block. In the event that `my_sbrk()` returns failure (by returning `-1`), you should set the error code `OUT_OF_MEMORY` and return `NULL`.

Remember that you want the address you return to be at the start of the block body, not the metadata. This is `sizeof (metadata_t)` bytes away from the very front of the block. Since pointer arithmetic is in multiples of the sizeof the data type, you can just add 1 to a pointer of type `metadata_t*` pointing to the front of the metadata to get a pointer to the body. If you have not specifically set the error code during this operation, set the error code to `NO_ERROR` before returning.

4. The first call to `my_malloc()` should call `my_sbrk()`. Note that malloc should call `my_sbrk()` when it doesn't have a block to satisfy the user's request anyway, so this isn't a special case.

3.8 my_free(void *ptr)

You are also to write your own version of free that implements deallocation. This means:

1. Calculate the proper address of the block to be freed, keeping in mind that the pointer passed to any call of `my_free()` is a pointer to the block's user data section and not to the block's metadata.
2. Attempt to merge the block with blocks that are next to it if those blocks are free. See the previous subsection on deallocating for more information on how to do this.
3. Finally, place the resulting block in the `addr_list` by setting the respective next pointer in each node for the `addr_list`. Do not forget to keep the list sorted by address.

Just like the `free()` in the C standard library, if the pointer is `NULL`, no operation should be performed.

3.9 my_realloc(void *ptr, size_t size)

You are to write your own version of realloc that will use your `my_malloc()` and `my_free()` functions.

1. `my_realloc()` should accept two parameters from the user, `void *ptr` and `size_t size`. It will attempt to effectively change the size of the memory block pointed to by `ptr` to `size` bytes, and return a pointer to the beginning of the new memory block.

2. Do **not** directly change the freelist or blocks in `my_realloc()` — leave that to `my_malloc()` and `my_free()`. This means you don't need to worry about shrinking or extending blocks in place[1]; if `size` is nonzero, just call `my_malloc()` to attempt to allocate a new block of the new size.

3. Make sure to copy as much data as will fit in the new block from the old block to the new block (don't forget to eventually free the old pointer). The rest of the data in the new block (if any) should be uninitialized.

Your `my_realloc()` implementation must have the same features as the `realloc()` function in the standard library. Specifically:

1. If the pointer is null, call `my_malloc` using the `size` argument (i.e. `my_malloc(size)`)

2. If the size is equal to zero, and pointer is non-null, call `my_free` using the ptr argument and return null (i.e. `my_free(ptr)`)

3. Else, create a new block via `my_malloc` and and copy the old block's data to the new block up to min(new block data size, old block data size)

4. If the requested size is nonzero and `my_malloc` fails, then do not free the old pointer.

Hint: Look at the man page for the C function memcpy

3.10 my_calloc(size_t nmemb, size_t size)

You are to write your own version of calloc that will use your `my_malloc()` function. `my_calloc()` should accept two parameters, `size_t nmemb` and `size_t size`. It will allocate a region of memory for `nmemb` number of elements, each of size `size`, zero out the entire block, and return a pointer to that block.

If `my_malloc()` returns `NULL`, do not set any error codes (as `my_malloc()` will have taken care of that) and just return `NULL` directly.

Hint: Look at the man page for the C function memset

3.11 Error Codes

For this assignment, you will also need to handle cases where users of your malloc do improper things with their code. For instance, if a user asks for 12 gigabytes of memory, this will clearly be too much for your 8 kilobyte heap. It is important to let the user know what they are doing wrong. This is where the enum in the `my_malloc.h` comes into play. You will see the three types of error codes for this assignment listed inside of it. They are as follows:

- **NO_ERROR**: set whenever `my_calloc()`, `my_malloc()`, `my_realloc()`, and `my_free()` complete successfully.
- **OUT_OF_MEMORY**: set whenever the user's request cannot be met because there's not enough heap space.

[1] Even though we don't extend or shrink blocks in place in this homework, keep in mind that real-world implementations (which are not written in a panic right before finals) very well could.

- **SINGLE_REQUEST_TOO_LARGE**: set whenever the user's requested size plus the total metadata size is beyond `SBRK_SIZE`. If both `OUT_OF_MEMORY` and `SINGLE_REQUEST_TOO_LARGE` occur, use `SINGLE_REQUEST_TOO_LARGE`. Ex. in the case of a request of 9kb, which is beyond our biggest block and total heap size, we set `ERRNO` to `SINGLE_REQUEST_TOO_LARGE`.

Inside the .h file, you will see a variable of type `enum my_malloc_err` called `my_malloc_errno`. Whenever any of the cases above occur, you are to set this variable to the appropriate type of error.

4 Running the Autograder and Debugging

The autograder must be run with the provided Docker container. To use the Docker container, open Docker Desktop, `cd` into the project directory and enter:

```
# macOS or Linux
./cs2110docker.sh

# Windows
.\cs2110docker.bat
```

If you are on macOS or Linux, you may get an error such as

```
zsh: permission denied: ./cs2110docker-c.sh
```

If so, make sure to enable execute permissions on the script: `chmod +x ./cs2110docker-c.sh`. You should only need to do this once.

Once you are in the Docker container, you can run the autograder:

```
# Clean your directory (remove any made files)
$ make clean

# Build and run all tests
$ make

# Build and run one test
$ make TEST=test_create_player::basic

# Build and run all tests in a single file
$ make TEST=test_create_player::*

# Build and run a test in GDB
# run-gdb takes the same parameters above
$ make run-gdb TEST=test_create_player::basic

# Build and run a test in valgrind
$ make run-valgrind TEST=test_create_player::basic
```

To test your code for memory leaks locally, you will need to use valgrind.

Note: There will be no checks with valgrind for Part 2 of the homework since you are implementing malloc and friends yourself, not using them! Therefore valgrind is only for Part 1.

The autograder format will look like the following:

```
[FAIL] test_create_player::duplicate_name: (0.01s)
[----] suites/create_player.c:111: Assertion Failed
[----]
[----]   Expected world player count to be correct
[----]
[----]   eq(i32, new_world->num_players, 1):
[----]     Actual: 0
[----]     Expected: 1
[FAIL] test_create_player::basic: (0.00s)
```

Here, the top value (Actual) represents what your solution produced and the bottom value (Expected) represents the correct value.

Note that the test cases are executed in parallel, so `[FAIL]` headers may not align exactly with the test failure message. If you want to test cases to run one by one, you can run `make test && ./tests -j1`.

4.1 Manual Testing

Note: This is for Part 1 only

If you want to write manual tests of your functions, you are allowed to modify `main.c` for use as a driver program. For example, you may want to create a new helper method that will print out the contents of the trainers array within `minecraft_v2.h`, implement it in `minecraft_v2.c`, and call it in `main.c`. However, if you choose to create extraneous helper methods to help debug **make sure to remove them when submitting to the autograder**. Editing `main.c`, however, should not interfere with the autograder.

Here is how to manually run your `main.c` code.

```
# Clean up all compiled output
$ make clean

# Recompile the driver executable
$ make driver

# Run the driver executable
$ ./driver
```

Important Notes:

1. Only the `.c` file will be graded on Gradescope.
2. All non-compiling solutions will receive a zero (with all the flags specified in the Makefile/Syllabus).
3. All solutions that do not meet syntax restrictions will also receive a zero.
4. **NOTE: DO NOT MODIFY THE HEADER FILES.**

   Since you are not turning them in, any changes to the `.h` files will not be reflected when running the Gradescope autograder.

We reserve the right to update the autograder and the test case weights on Gradescope or the local checker as we see fit when grading your solution.

5 Deliverables

Please upload the following files to Gradescope:

1. `minecraft_v2.c`
2. `my_malloc.c`

6 Appendix

7 Rules and Regulations

1. Please read the assignment in its entirety before asking questions.
2. Please start assignments early, and ask for help early. Do not email us the night the assignment is due with questions.
3. If you find any problems with the assignment, please report them to the TA team. Announcements will be posted if the assignment changes.
4. You are responsible for turning in assignments on time. This includes allowing for unforeseen circumstances. If you have an emergency please reach out to your instructor and the head TAs **IN ADVANCE** of the due date with documentation (i.e. note from the dean, doctor's note, etc).
5. You are responsible for ensuring that what you turned in is what you meant to turn in. No excuses if you submit the wrong files, what you turn in is what we grade. In addition, your assignment must be turned in via Gradescope. Email submissions will not be accepted.
6. See the syllabus for information regarding late submissions; any penalties associated with unexcused late submissions are non-negotiable.

7.1 Academic Misconduct

Academic misconduct is taken very seriously in this class. Quizzes, timed labs and the final examination are individual work. Homework assignments will be examined using cheat detection programs to find evidence of unauthorized collaboration.

You are expressly forbidden to supply a copy of your homework to another student. If you supply a copy of your homework to another student and they are charged with copying, you will also be charged. This includes storing your code on any platform which would allow other parties to it (public repositories, pastebin, etc). If you would like to use version control, use a private repository on github.gatech.edu

Homework collaboration is limited to high-level collaboration. Each individual programming assignment should be coded by you. You may work with others, but each student should be turning in their own version of the assignment.

High-level collaboration means that you may discuss design points and concepts relevant to the homework with your peers, share algorithms and pseudo-code, as well as help each other debug code. What you shouldn't be doing, however, is pair programming where you collaborate with each other on a single instance of the code, or providing other students any part of your code.

Submissions that are essentially identical will receive a zero and will be sent to the Dean of Students' Office of Academic Integrity. Submissions that are copies that have been superficially modified to conceal that they are copies are also considered unauthorized collaboration.

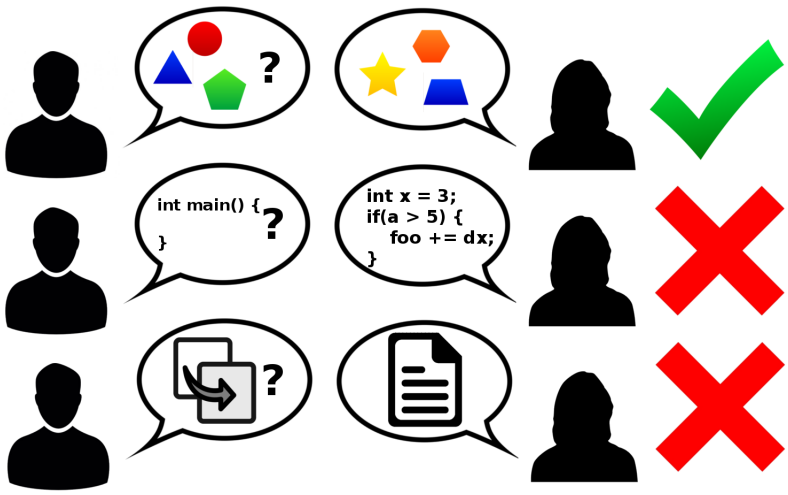


Figure 1: Collaboration rules, explained colorfully